[code_analysis] Query Processing Parser-Semantic Check Cheat Sheet

Created By	Hyung-Gyu Ryoo
Email	hgryoo@cubrid.com
Tags	코드분석
Team	개발2팀
Version	1.0

이 자료는 Parser, Prefetch와 Semantic check 단계에서 각 주요 함수들을 빠르게 찾아갈 수 있도록 돕기 위한 문서입니다. 각 단계, 또는 함수별로 어떤 동작을 하는지는 소스코드 공유 문서 또는 분석서를 참고해주세요.

High Level API
메인 로직
- Prefetch 주요 함수들
- Semantic check 주요 함수들
  - Resolve names (PT_NAME 관련 모든 로직)
  - Semantic check local (check semantics, type checking, constant folding)

High Level API

/* Client로부터 실행 */

```
ux_prepare()
ux_execute()
```

/* CSQL로부터 실행 */

```
csql_execute_statements()
```

메인 로직

Parsing, Compile 과정

```
parser_main()

/* db_vdb.c SQL을 컴파일하기 위한 모든 단계를 포함하는 함수 */
db_compile_statement_local()

  /* Compilation stage */
  pt_class_pre_fetch()
  pt_compile(PARSER_CONTEXT *parser, PT_NODE *statement) // Semantic check 관련 부분
  mq_translate() // view transform, rewrite
  pt_class_pre_fetch()

  /* Preparation stage */
  do_prepare_statement(PARSER_CONTEXT *parser, PT_NODE *statement)
```

Prefetch 주요 함수들

```
/* compile.c */
pt_class_pre_fetch ()
  /* locator_cl.c */
  locator_lockhint_classes ()
    if (need_call_server == true)
    {
      /* network_interface_cl.c
      *  LC_LOCKHINT, LC_COPYAREA
      */
      locator_find_lockhint_class_oids ()
    }
```

Semantic check 주요 함수들

```
pt_compile()
  pt_semantic_check()
    pt_check_with_info()

    switch (node->node_type)
    {
      ...
      case PT_SELECT:
         /* PT_NAME resolution */
         pt_resolve_names ()
         /* aggregate/analytic function이 where에 있는지 확인 */
         pt_check_where ()
         /* replace host variable (?) into PT_VALUE */
         node = parser_walk_tree (parser, node, pt_check_and_replace_hostvar, &check, pt_continue_walk, NULL);
         /* check semantics, type checking and constant folding */
         node = parser_walk_tree (parser, node, NULL, NULL, pt_semantic_check_local, sc_info_ptr);
         /* path expression 관련, 분석 생략 */
         node = parser_walk_tree (parser, node, pt_expand_isnull_preds, node, NULL, NULL);
         break;
      ...
    }
```

Resolve names (PT_NAME 관련 모든 로직)

```
pt_resolve_names()

  /* entity spec을 flat list로 바꿈 */
  statement = parser_walk_tree (parser, statement, pt_flat_spec_pre, &info, pt_continue_walk, NULL);
  /* name binding 함수 */
  statement = parser_walk_tree (parser, statement, pt_bind_names, &bind_arg, pt_bind_names_post, &bind_arg);
  /* resolve alias in group by/having */
  statement = parser_walk_tree (parser, statement, pt_resolve_group_having_alias, NULL, NULL, NULL);
  /* natural join을 inner/outer join으로 변환 */
  statement = parser_walk_tree (parser, statement, NULL, NULL, pt_resolve_natural_join, NULL);
  /* SELECT ... FOR UPDATE */
  if (statement != NULL && statement->node_type == PT_SELECT
      && PT_SELECT_INFO_IS_FLAGED (statement, PT_SELECT_INFO_FOR_UPDATE))
  {
    ...
  }
```

```
pt_flat_spec_pre()

  if (node->node_type == PT_SPEC)
  {
    if (!node->info.spec.flat_entity_list && PT_SPEC_IS_ENTITY (node))
    {
      node->info.spec.id = (UINTPTR) node;
      q = pt_make_flat_name_list (parser, node, info->spec_parent, info->for_update);
      node->info.spec.flat_entity_list = q;
    }

    // derived table인 경우
    parser_walk_tree (parser, derived_table, pt_flat_spec_pre, info, pt_continue_walk, NULL);
  }
```

```
pt_bind_names()

  switch (node->node_type)
  {
    ...
    case PT_SELECT:
      /* CTE 관련 */
      pt_bind_names_in_with_clause(parser, node, bind_arg);
      /* derived table 먼저 bind_names */
      pt_bind_scope()

      /* 여러가지 SQL Hint를 확인 */
      line 1709~

      /* '*' 또는 'class_name.*'를 풀어내기 위한 코드 구현 영역 */
      line 1767~

      /* ORACLE OUTER JOIN이 있는 경우 rewrite (더 분석 필요)*/
        /* where 절을 cnf 형태로 바꿈 */
        node->info.query.q.select.where = pt_cnf (parser, node->info.query.q.select.where);
        /* where 절의 join 조건과 FROM 절을 매칭하여 spec 찾기 */
        /* ANSI join으로 rewrite */

    case PT_NAME:
      /* PT_NAME 노드를 scope 내의 spec과 연결 */
      pt_bind_name_or_path_in_scope()
    case PT_SPEC:
      /* location 값 세팅 */
      /* entity가 system_class 인지 확인 */
    ...
  }
```

```
pt_resolve_group_having_alias() 에서 주요 부분, 다른 부분은 PT_NAME을 찾기 위한 순회
:SELECT list의 column의 alias와 group by의 name을 매칭, 이름 재할당

//pt_resolve_group_having_alias_pt_name에서 주요 로직 실행, expr, sort_spec은 name을 찾기 위한 부분
for (col = select_list; col != NULL; col = col->next)
    {
      if (col->alias_print != NULL && intl_identifier_casecmp (n_str, col->alias_print) == 0)
  {
    parser_free_node (parser, *node_p);
    *node_p = parser_copy_tree (parser, col);
    if ((*node_p) != NULL)
      {
        (*node_p)->next = NULL;
      }
    break;
  }
    }
```

Semantic check local (check semantics, type checking, constant folding)

```
pt_semantic_check_local()

  switch (node->node_type)
  {
    ...
    case PT_SELECT:
      /* illegal multi-column subquery를 검사
         (로직은 있지만 parser에서 검사됨) */
      pt_length_of_select_list()

      pt_check_into_clause()

      if (pt_has_aggregate (parser, node))
      {
        /* aggregate 관련 검사 */
        // PPT 참고
      }

      if (pt_has_analytic (parser, node))
      {
        /* analytic 관련 검사 */
        // PPT 참고
      }

      if (pt_check_order_by (parser, node))
      {
        /* order by 관련 검사 */
      }

      /* CONNECT BY 관련 (hierarchical query) */
      if (node->info.query.q.select.connect_by)
      {
         ...
      }

      /* SHOW statement의 metadata로 select list를 rewriting */
      check_func = showstmt_get_metadata ()
      node = (*check_func) (parser, node);

      /* type checking and constant folding */
      node = pt_semantic_type (parser, node, info);
      break;
    ...
  }
```

```
pt_semantic_type()

  /* do type checking */
  tree = parser_walk_tree (parser, tree, pt_eval_type_pre, sc_info_ptr, pt_eval_type, sc_info_ptr);
  /* do constant folding */
  tree = parser_walk_tree (parser, tree, pt_fold_constants_pre, NULL, pt_fold_constants_post, sc_info_ptr);

/*==========================================*/
  pt_eval_type_pre()
  switch (node->node_type)
  {
    case PT_SPEC:
    /* outer join이 존재하는지 체크 */

    case PT_SORT_SPEC:
    /* sort spec expression이 query인지 체크 */

    case PT_SELECT:
```

```
    /* LIMIT rewriting */

    case PT_EXPR:
    /* recursive expression 처리 */
  }

  pt_eval_type()
    eval_expr_type() /* PT_EXPR */
    eval_function_type() /* PT_FUNCTION */
    where_type() /* PT_SELECT */
/*==========================================*/
    eval_expr_type() /* PT_EXPR */
        /* 모든 expression op에 대해서 다룰 순 없다.
           필요한 경우 해당 expression이 함수 내에서 지나가는 부분을 확인 */

          /* 룰 기반의 type evaluation (op에 따라 implicit conversion table 존재)*/
          switch (op)
          {
            ..
            case PT_PLUS:
            ...
          }
          /* 미리 정의한 expression definition에 따라 evaluation (explicit conversion table) */
          pt_apply_expressions_definition(parser, &expr)
            pt_get_expression_definition() // operator마다 정의한 expression definiion
            //best_match => 가장 잘 맞는 expression definition 검색
            pt_coerce_expr_arguments() // 필요하면 타입 변환

          /* Collation을 가진 타입(String)의 경우 compatible 확인 */
          pt_check_expr_collation()
          /* PT_TYPE_MAYBE 타입 wrapping 여부 결정 */
          pt_wrap_expr_w_exp_dom_cast()

          /* 결과 타입이 자명한 경우에 대해 type evaluation (e.g. TO_CHAR) */
          switch (op)
          {
            ...
          }
```